

Garbage Collector Tuning Harness for DaCapo-Chopin

DIMITRIS ASIMAKOPOULOS - HY446

Motivation

- explore JVM flags and how they can dramatically shift results
- identify the true “sweet spots” of different garbage collectors
- use the state-of-the-art and versatile suite, DaCapo-Chopin
- familiarize with scripting, automation and regular expressions
- get insight into gc’s architecture by reading logs
- evaluate correctly and follow the rules

-Xmx/-Xms	Controls the total memory available. Larger heaps can reduce frequency of GC but may increase pause durations.	**all**
-XX:MaxGCPauseMillis	Hints G1's pause-time goal. Lower values drive more concurrent work (higher CPU) to hit sub-N ms pauses. Higher values allow more throughput.	**G1**
-XX:ParallelGCThreads	Number of threads mainly for young collections. More threads can shorten GC pause times at cost of CPU.	**Parallel, G1, ZGC**
-XX:ConcGCThreads	Threads for concurrent phases (mark, cleanup). More threads speed concurrent work, reducing "stop-the-world" time spent waiting on concurrent tasks.	**G1, ZGC**
-XX:MaxTenuringThreshold	How many times an object can survive minor GC before promotion to old gen. Higher values keep objects longer in Survivor space.	**Parallel, G1**
-XX:+AlwaysPreTouch	Forces RAM pages to be faulted in at startup ("pre-touched"), smoothing out later page-fault latencies at cost of longer startup.	**all**
-XX:+UseLargePages	Enables huge pages to reduce TLB misses and kernel overhead, improving latency predictability.	**all (if os supports)**
-XX:SurvivorRatio	Ratio between Eden and each Survivor space. Affects promotion frequency and tenuring.	**Parallel, G1**
-XX:+UnlockExperimental VMOptions -XX:+UseShenandoahGC	Turns on Shenandoah's ultra-low pause collector. Great for sub-millisecond pauses in large heaps. use when G1/ZGC pauses prove too long.	**Shenandoah**

Methodology (1/3)

```yaml
- name: g1-4g-p200-t8-c4
  gc: G1GC
  xmx: 4g
  pause: 200
  pgc: 8
  cgc: 4
  survivorRatio: 8
  maxTenuringThreshold: 15
  pretouch: false
  largePages: false

- name: g1-8g-p50-t4-c2
  gc: G1GC
  xmx: 8g
  pause: 50
  pgc: 4
  cgc: 2
  survivorRatio: 8
  maxTenuringThreshold: 15
  pretouch: false
  largePages: false
```

choose garbage collectors

- **Parallel**: the traditional parallel-stop-the-world collector optimized for maximum application throughput. Serves as baseline.
- **G1**: HotSpot's default, balances pause-time goals with overall throughput via concurrent marking and regionized heap. Represents a modern GC.
- **ZGC**: low-pause, concurrent collector designed to scale to large heaps with sub-millisecond pause targets. Lets us explore ultra-low-latency.

select versatile benchmarks

run 22 DaCapo-Chopin benchmarks to cover web & middleware (tomcat, tradebeans, spring), compilers (eclipse, pmd), graphics (batik, sunflow, jme) databases(cassandra, h2, kafka), machine learning (h2o, biojava) and more.

define the workload universe

generate a Cartesian "configs.yaml" of the most significant flag combinations for Parallel, G1, ZGC.

Methodology (2/3)

automate execution harness

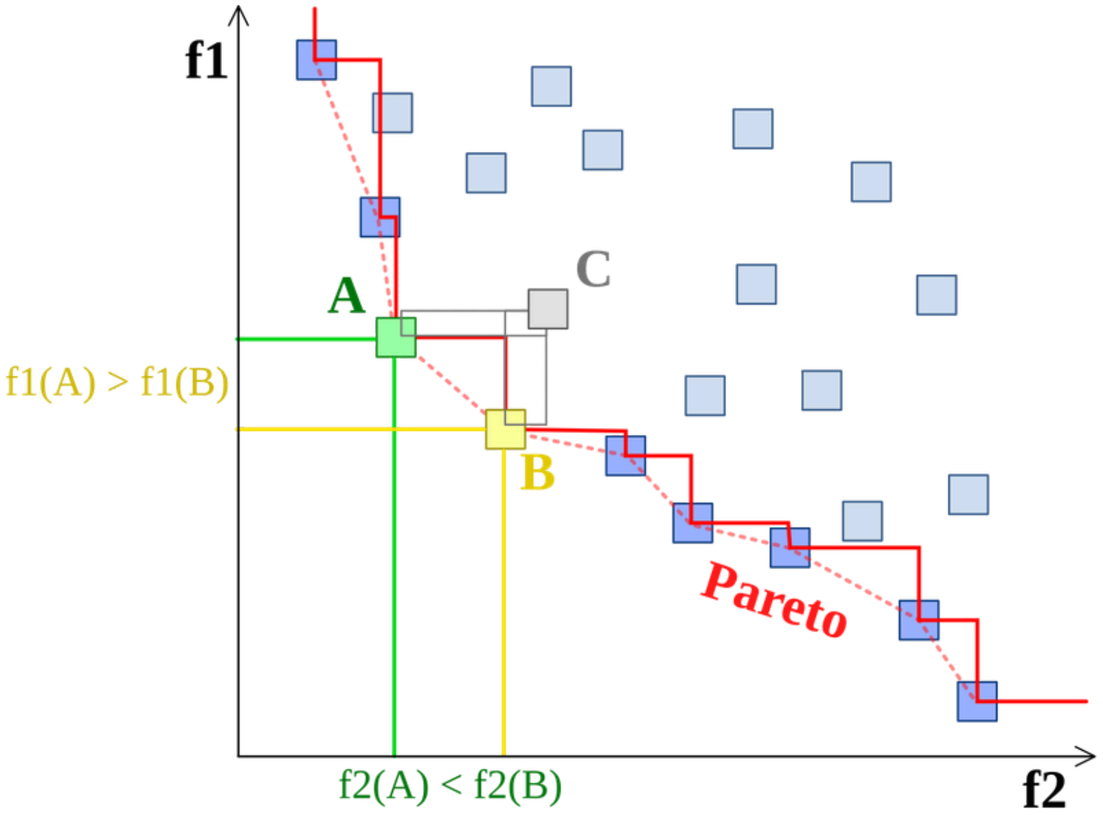
- create run_benchmarks.sh script that loops over <benchmark,config> pairs
- launch dacapo-chopin.jar for 10 iterations for every benchmark
- capture stdout into output.txt and GC traces into gc.log
- ubuntu 20.04, 16gb RAM, i7-3820 CPU@3.60GHz × 8

parse logs and extract important metrics

- regex-scan each gc.log for pause statistics (min/median/p99/max), total GC time, GC counts, to-space events and output.txt for the total runtime
- save these metrics into results/metrics.csv
- debug and repeat 3 times for more accurate results
- categorize by config and average all metrics using python

```
benchmark,config,total_time_ms,total_gc_ms,pct_gc_time,min_pause_ms,p50_ms,p99_ms,max_pause_ms,young_gc_count,full_gc_count
avrora,parallel-4g-t4,11457.0,136.3,1.19,0.4,6.6,18.2,18.2,10,10
avrora,parallel-4g-t8,11463.0,105.1,0.92,0.4,5.8,11.9,11.9,10,10
avrora,parallel-8g-t4,11448.0,137.1,1.20,0.3,7.1,18.3,18.3,10,10
avrora,parallel-8g-t8,11798.0,118.3,1.00,0.3,6.6,14.3,14.3,10,10
avrora,g1-4g-p50-t4-c2,11605.0,179.3,1.54,15.6,17.5,23.5,23.5,0,10
avrora,g1-4g-p50-t4-c4,11480.0,176.9,1.54,16.7,17.4,19.5,19.5,0,10
avrora,g1-4g-p50-t8-c2,11508.0,158.2,1.37,15.1,15.8,16.6,16.6,0,10
avrora,g1-4g-p50-t8-c4,11231.0,165.1,1.47,14.6,15.2,26.9,26.9,0,10
avrora,g1-4g-p200-t4-c2,11329.0,172.3,1.52,15.6,16.4,22.2,22.2,0,10
avrora,g1-4g-p200-t4-c4,11911.0,181.2,1.52,15.7,17.2,25.6,25.6,0,10
```

Methodology (3/3)



- **quantitative analysis**
  - compute derived metrics (e.g. GC overhead %, allocation rate, memory reclaimed)
  - generate plots of pause distributions per GC & heap size
  - highlight Pareto-optimal points
  - summarize best-in-class configurations

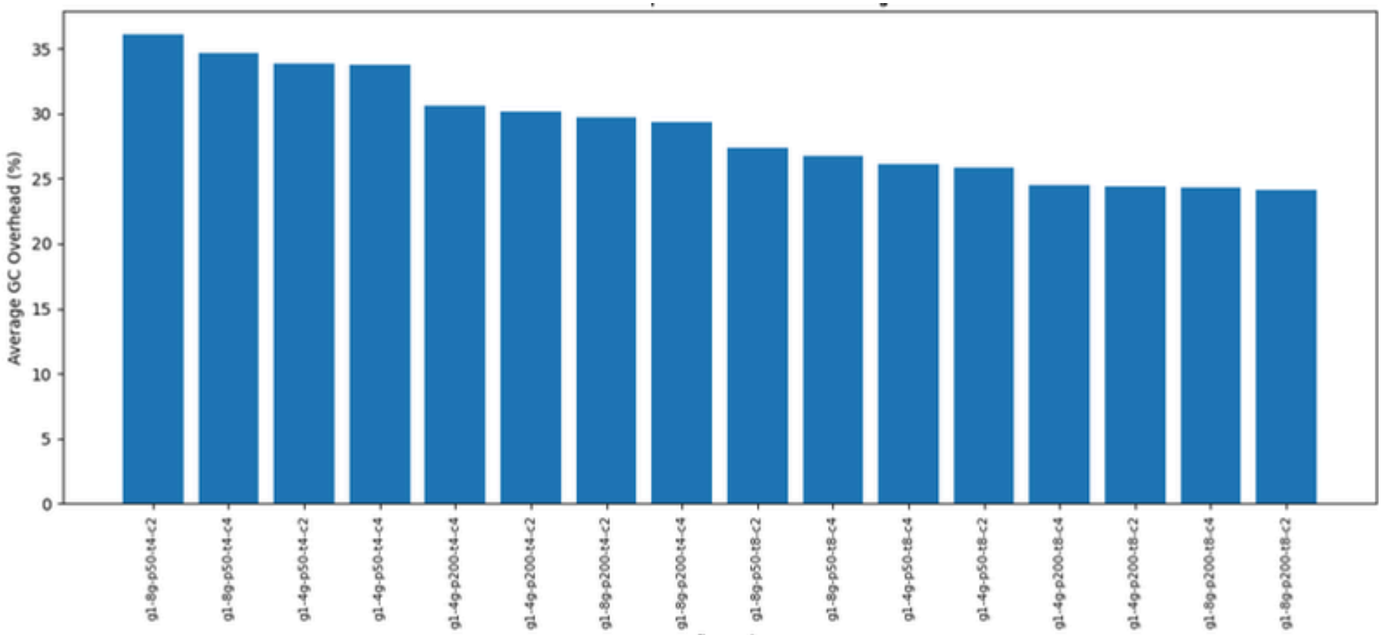
- a point is Pareto-optimal if you cannot improve one metric (e.g. runtime) without worsening the other (e.g. pause time). The Pareto frontier is the set of all such non-dominated points. It traces out the best trade-offs you can achieve.

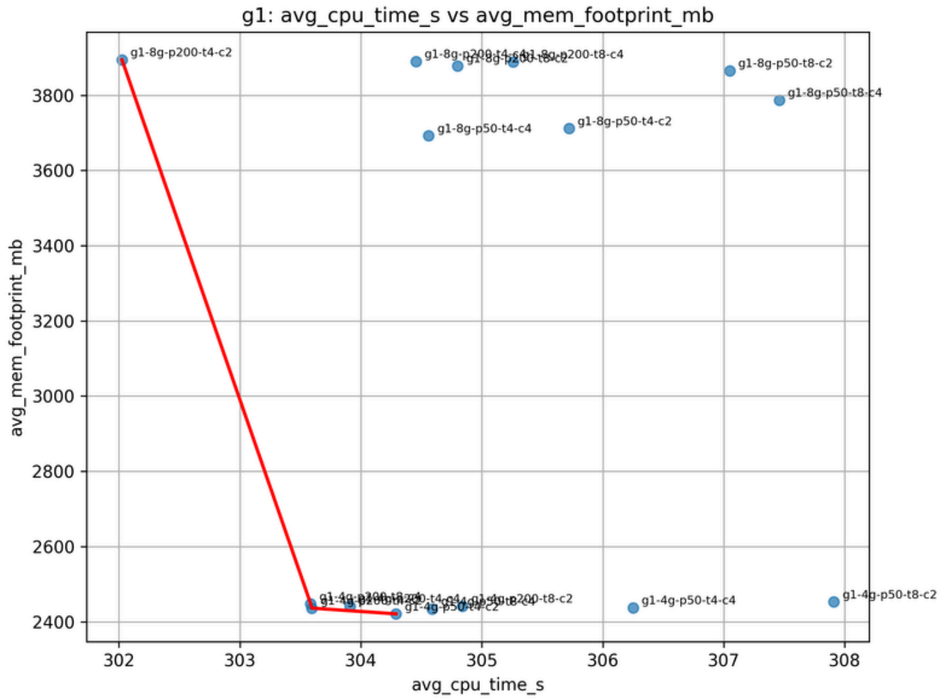
G1 Results

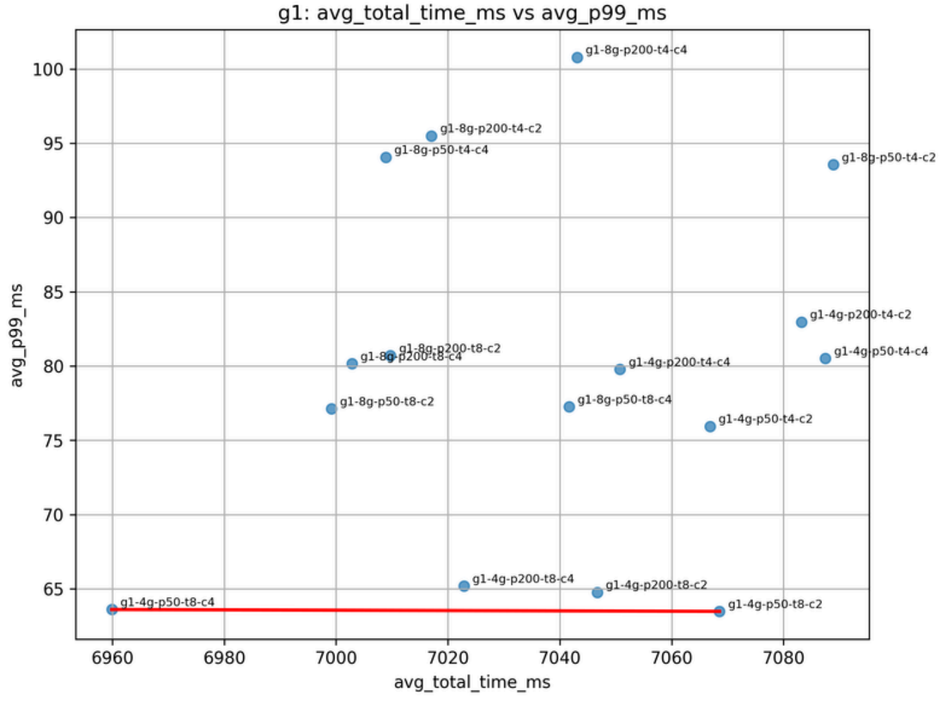
heap	pause	parallel	concurrent	throughput (ms)	overhead (%)	total gc time (ms)	cpu time (s)	memory (mb)
4	50	4	2	7050.35	33.839	1711.18	304.29	**2420.975**
4	50	4	4	7021.55	33.744	1687.66	306.25	2437.41
4	50	8	2	6996.7	25.807	1241.49	**307.91**	2452.7
4	50	8	4	7017.15	26.072	1248.78	304.59	2434.28
4	200	4	2	6992.75	30.117	1409.94	303.595	2436.23
4	200	4	4	7028.45	30.576	1406.85	303.91	2441.845
4	200	8	2	**6987.75**	24.389	1091.21	304.84	2441.605
4	200	8	4	6996.65	24.477	1090.83	303.585	2447.75
8	50	4	2	7004.1	**36.055**	**1752.6**	305.72	3711.34
8	50	4	4	6999.05	34.692	1713.83	304.56	3692.655
8	50	8	2	7019.5	27.378	1274.40	307.05	3865.325
8	50	8	4	7010.8	26.712	1245.40	307.46	3787.35
8	200	4	2	6993.9	29.686	1272.6	**302.025**	**3894.24**
8	200	4	4	7017.25	29.362	1257.99	304.455	3890.115
8	200	8	2	**7229.211**	**24.127**	1010.64	304.8	3878.41
8	200	8	4	7001.85	24.286	**997.53**	305.26	3888.82

G1 Performance Conclusions

- more parallel threads offer better throughput & lower GC overhead
- tighter pause goal (50 ms) lowers high-percentile pauses, at a small throughput cost
- larger heap (8 GB) doesn’t help throughput in this range
- concurrent thread count has minor impact
- CPU usage is remarkably stable across all G1 configurations, varying by only a few percent
- tighter pause goals sometimes edge CPU time slightly upward, reflecting extra work to meet latency constraints.
- increasing parallel or concurrent thread counts has minimal impact on overall CPU consumption
- doubling the heap from 4 GB to 8 GB raises the memory footprint by roughly half again, with only marginal CPU-time benefits
- pause-time and thread adjustments tune latency without significantly altering compute or memory costs





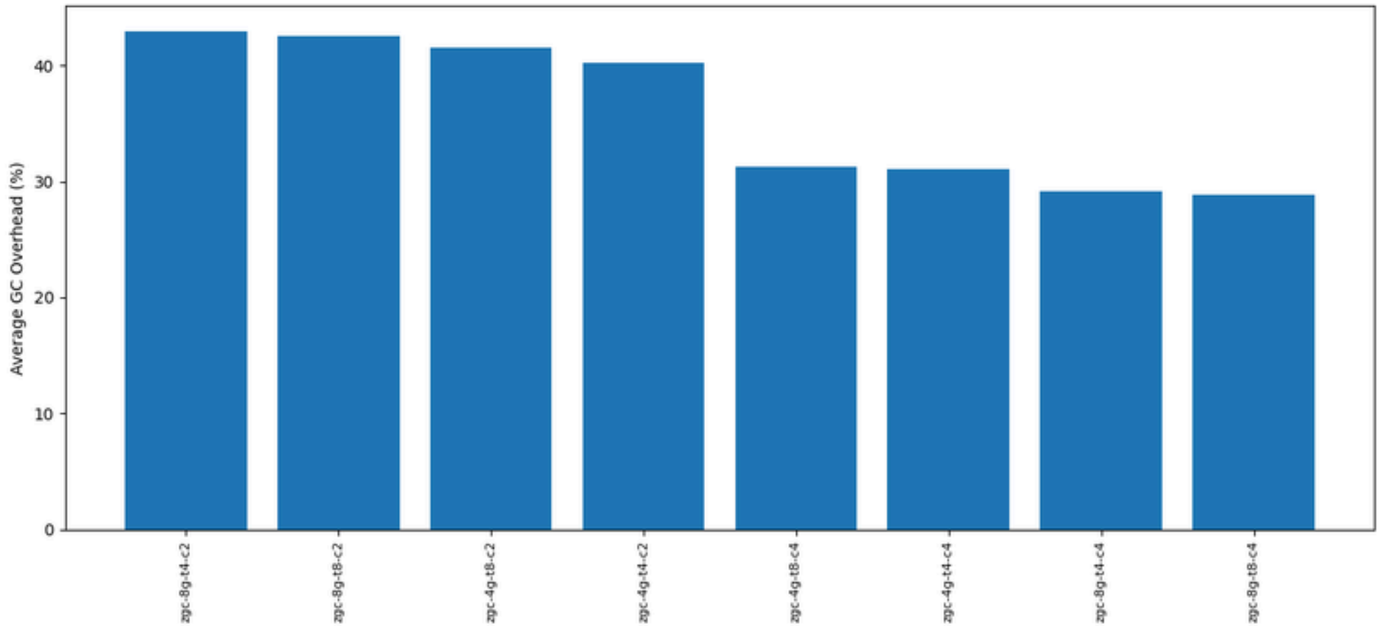


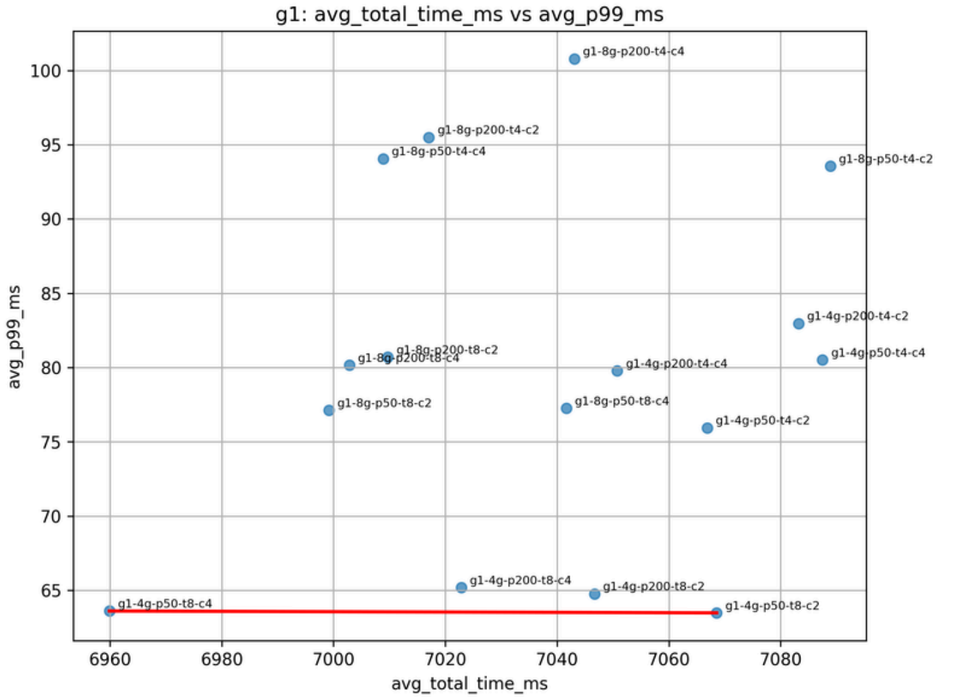
ZGC Results

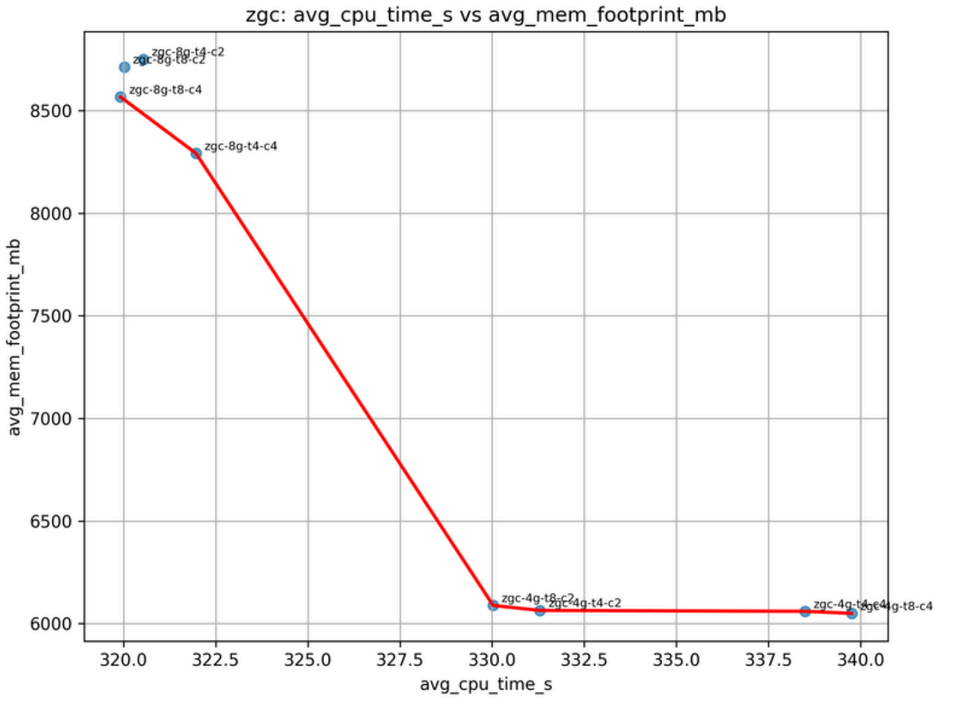
heap size	parallel threads	concurrent threads	throughput (ms)	overhead (%)	total gc time (ms)	p99 pause times	gc events	cpu time (s)	memory footprint (mb)
4	4	2	**7471.65**	40.223	3243.255	257.01	10.5	331.3	6064.14
4	4	4	7352.15	31.042	2323.73	232.995	10.5	338.49	6059.925
4	8	2	7436.2	41.505	**3260.065**	271.49	10.5	330.03	6088.42
4	8	4	7362.05	31.243	2368.985	**226.2**	10.5	**339.755**	**6049.905**
8	4	2	7183.4	**42.983**	3063.09	**271.75**	10.5	320.54	**8749.43**
8	4	4	7197.95	29.116	2055.075	235.575	10.5	321.965	8291.29
8	8	2	**7150**	42.52	3000.47	258.995	10.5	320.02	8712.54
8	8	4	7176.2	**28.89**	**2036.7**	241.66	10.5	**319.92**	8565.985

ZGC Performance Conclusion

- concurrent threads have the biggest impact on overhead
- larger heap (8 GB) reduces both total time and GC overhead
- increasing parallel threads helps only on the larger heap
- memory usage scales with heap size, confirming ZGC's resident set tracks the configured heap
- smaller heaps drive up overall CPU consumption compared to larger heaps
- adding more concurrent threads only slightly shifts CPU load, with effects varying by heap size
- ZGC relies heavily on CPU cycles relative to wall-clock time to achieve its low-pause design





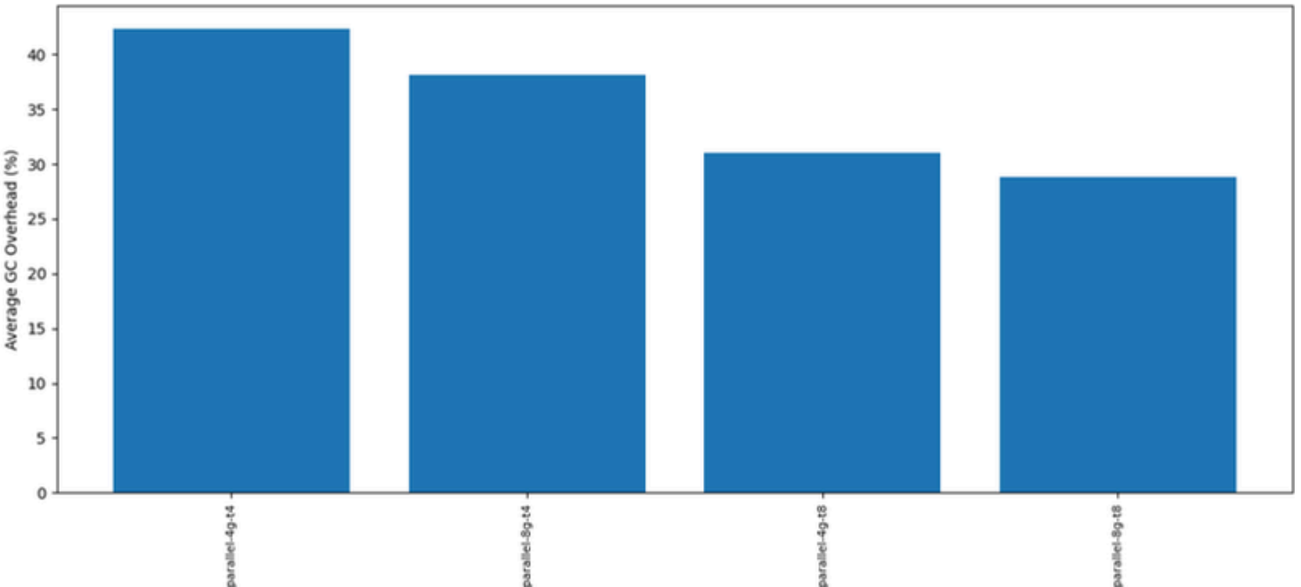


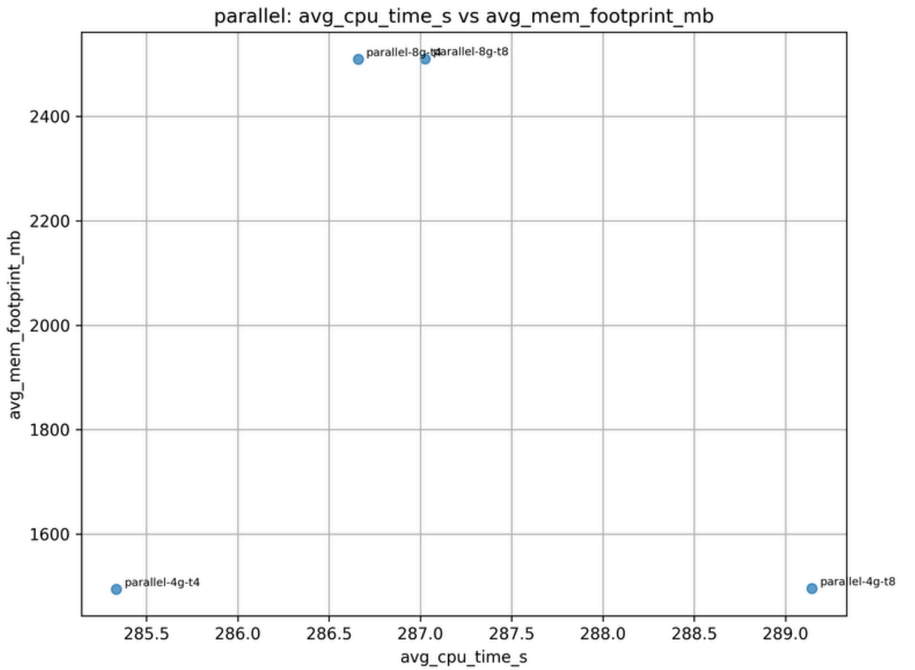
Parallel Results

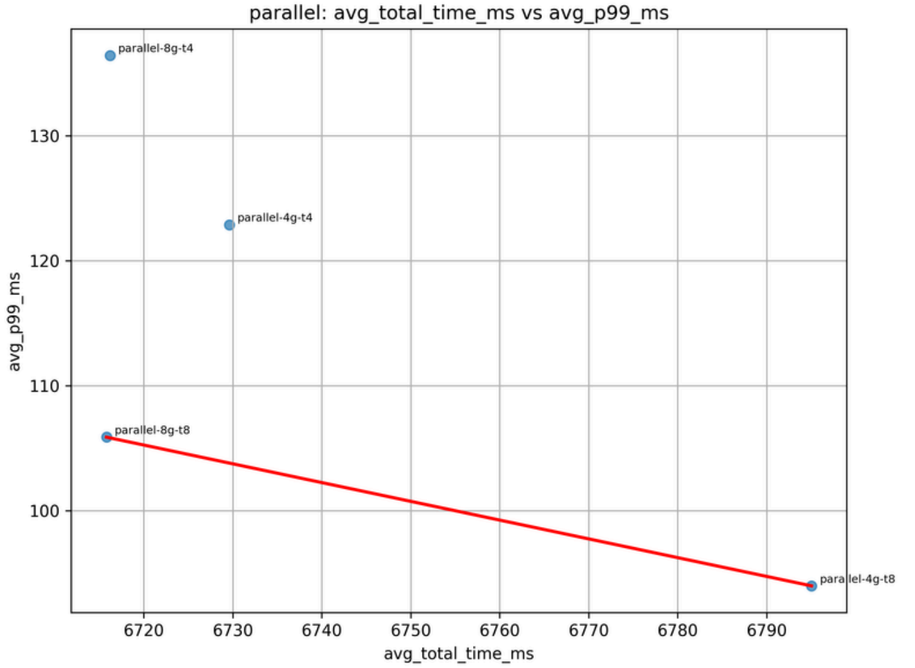
heap size	parallel threads	throughput (ms)	overhead (%)	total gc time (ms)	p99 pause times	gc events	cpu time (s)	memory (mb)
4	4	6735.85	42.3	1663.665	114.57	**75**	**285.335**	**1494.33**
4	8	**6719.6**	30.987	1231.68	**92.24**	74.3	**289.145**	1495.475
8	4	6721.95	**38.113**	1519.265	**135.67**	**46.5**	286.66	2509.49
8	8	**6756.75**	**28.858**	**1178.54**	103.845	46.6	287.025	**2510.455**

ParallelGC Performance Conclusion

- parallel threads cut both runtime and GC overhead
- heap size increase yields diminishing returns
- best config is 4gb heap and 8 threads
- changing the number of GC threads has virtually no effect on application CPU time.
- doubling the heap size roughly doubles the memory footprint while leaving CPU time nearly unchanged.
- gc configuration tweaks shift memory usage far more than they shift compute cost.
- keeping a smaller heap delivers similar compute performance at a significantly lower memory cost.







Interesting Benchmark-Specific Observations

- ZGC configurations for **h2** show extreme GC times (e.g., zgc-4g-t4-c2 has total_gc_ms=42,050.6) meaning that ZGC may not handle high-memory-pressure applications like H2 efficiently, possibly due to allocation rates or heap sizing issues.

- ZGC configurations (e.g., **avrora**,zgc-4g-t4-c2) consistently show min_pause_ms=0.0, aligning with its design for ultra-low pause times. This makes ZGC suitable for latency-sensitive applications, as seen in benchmarks like **fop** where ZGC outperforms others.

- while ZGC excels in **fop**, it performs poorly in **tradebeans**, indicating its effectiveness depends heavily on application behavior and memory access patterns.

Interesting Benchmark-Specific Observations

- benchmarks like **pmd** with Parallel GC show high full_gc_count, leading to long pauses (e.g., p99_ms=82.0). This underscores Parallel GC's limitations in managing fragmented heaps for certain workloads.

- workloads like **lusearch** with Parallel GC have high young_gc_count (e.g., 173) but low full_gc_count, indicating efficient young-gen collection but potential inefficiency in old-gen management, leading to frequent minor pauses.

- configurations like **biojava**, g1-4g-p50-t4-c2 vs. p200-t4-c2 show minimal differences in total time, suggesting pause targets (p50, p200) may not significantly impact performance for some workloads.

Lower Bound Overhead

- for each run we calculate **app_time = avg_total_time_ms − avg_total_gc_ms**
- find the minimum **app_time** across all configurations
- for any config: **LBO_overhead_pct = 100 × (avg_total_time_ms − min_app_time) / avg_total_time_ms**

Best Configurations	Measured GC%	LBO Overhead%
g1-4g-p200-t8-c2	24.39 %	21.47 %
g1-4g-p50-t8-c2	25.81 %	21.57 %
parallel-4g-t8	30.99 %	18.33 %
parallel-8g-t8	28.86 %	17.45 %
zgc-8g-t8-c4	28.89 %	28.40 %

A lower LBO Overhead % means we’re closer to the ideal zero-GC limit.

thank you!